The Thiele Machine

A Mathematical Reference

Devon Thiele

Before the specification. I don't do math by hand. I think in shapes, directions, and connections, and my part of this work was to see where things had to fit, to direct, and to refuse what didn't fit. This document is the translation. Language models wrote the formal statements and the proofs, and I read the statements to make sure they say what the shape says. Coq checks the formal counterparts identified below; some on-page arguments have no Coq counterpart here, as noted where they occur. The arguments that carry the results are in the monograph, on the page; the formal versions here are their counterparts, and compiling them is a tool for catching mistakes, not the reason to believe them. So nothing below is in my voice, on purpose. It's the reference layer: exact statements, exact premises, and where each one lives. If you want the pictures, the monograph starts with them. If you want to know exactly what is claimed, it's here.

The Thiele Machine is an abstract model of computation, and an argument made with it about what an account of computation should preserve.

It is a machine in the sense a Turing machine is one: a mathematical object. No device is meant. The model starts from a set of states, moves, a step, a price on each move, a record that can be compared, and one rule, the toll: a move that takes the record out of the down-set of where it stood pays at least one. Any system with those parts is a priced record system; a Thiele machine is one whose record some run raises, from a designated start, and no move lowers. A yes-or-no reading is the smallest record, and most of what follows is stated on it. A reading no move carries out of its down-set is flat, and the toll asks nothing of it: a clock, a reversible process and every ordinary deterministic machine, read through a reading their moves never raise, are flat; each, once its states can be tested for equality and it has a run that leaves its start, is the shadow of a member. Forget the prices and the record of any Thiele machine and what is left is one of them; the results below are about what that forgetting throws away. The latch, two states and one priced move, shows the definition can be met. It is not the model.

The structural question is what a total deterministic process needs to raise a reading and keep it: a merge along the run or endlessly fresh states. A physical price needs separate hypotheses. Landauer bounds depend on the physical model and input distribution; finite-time entropy-production bounds here concern specified thermal stochastic dynamics, not every physical state change. Certification, a record that a claim was checked and closed, is the sharpest case. It is one example of a wider question about structural state, observation and cost. A window, a function from states to what can be seen of them (Chapter 5), can lose distinctions that the full state keeps, such as the record and the ledger; the cost laws of Chapters 3 and 4 constrain specified events under hypotheses each theorem names. A conventional encoding can preserve those distinctions. A window that discards them cannot recover them in general. The stronger proposal that these distinctions belong in a necessary foundation of computation is a conjecture. The status of each result is stated below.

Abstract

This is a mathematical reference for the Thiele Machine: a deterministic transition system with instruction prices and a record that is a position in a preorder, with a toll on every step that moves the record out of the down-set of where it stood, a record no step lowers, and a designated start from which some run raises it. The one-bit record, a Boolean reading with its false-to-true crossing, is the two-point instance, and most results are stated on it. An internal ledger is an additional condition used by particular results. It states the model, its toll, the member predicate, the smallest member and the history lift, then the results, each over the class its premises name. Each result is stated with its premises and names the declaration that carries it.

A formal proof shows that conclusions follow from premises. It does *not* show that premises model physical reality. Every result is tagged as one of three types: **(S)** structural theorems about the model or a stated machine, **(C)** conditional results that hold under a named premise no machine fact discharges, and **(R)** refutations, where a theorem states the negation of a proposed claim and a named construction supplies the counterexample. One proposal, the pointer criterion of Chapter 9, has no proof; it carries a fourth tag, thesis.

The structural results (S), by where they sit in the model:

- *The definition and the price in the move.* The member predicate: the toll, permanence and a run that makes a distinction; a flat reading obeys the toll and is not a member; the latch is the smallest member; every system with a Boolean equality on states and a run that ends off its start is the shadow of a member. The floor is the toll over any order of records, and the universal floor on the one-bit instance. The least instruction price and the least price for the lift. On finite machines, the toll derived from merge pricing for permanent certificates. Shadow pricing: no price computed from a window that hides a crossing is exact.

- *What stays.* A free move cannot touch a permanent reading. The record axis over any base is a latch, and a growing record decomposes into threshold latches.

- *What a window forgets.* The coarsest observational identification, and the factorization of histories through it. Shadow pricing, and what a window can decide and price.

- *The room a kept distinction needs.* On any state space a permanent flip merges or needs a fresh state every time, with the exact retention bound for a bounded memory, and the stochastic and unitary forms of the same count.

- *What no inside procedure decides.* The structural shortcut wall over any substrate with a recursion theorem; the recursion theorem and Rice's theorem (the logician Henry Rice, 1953: no program decides a property of what programs do, unless it holds of every program or of none) for L.

That a flat reading, a toggle and a one-state system are not members, and the counterexamples that show which premises do work, are refutations (R). Results stated over Landauer heat are conditional (C) on the named premise `landauer_heat`.

Contents

1 Introduction **2**

2 Notation and Conventions **3**

3 Thiele Machines and the Universal Floor **4**
3.1 Free moves, least prices and observation 6
3.2 A2 among cost frameworks 8
3.3 Several floors and one charge 8

4 Finite Machines and the Price of Merging **9**
4.1 Entropy and heat 12
4.2 An eight-state instance 12
4.3 No intrinsic joule value 12

5 Windows, Pricing and Narrowing **14**

6 Recursion and Undecidability **16**

7 The Record Axis **17**
7.1 Growing and probabilistic records 17
7.2 Bases 18
7.3 The record over any order 18

8 The Physical Boundary **19**

9 External Systems and the Pointer Thesis **20**
9.1 The pointer criterion 20

A Proof File Index **22**

B Epistemological Classification **24**

Chapter 1

Introduction

A Thiele machine has states, moves, a price on each move and a record in a preorder; a move that raises the record pays (the toll), no move lowers it, and some run from a designated start raises it. A ledger, where a result uses one, is an additional condition, and a window that hides the record and the ledger is the subject of Chapter 5. The chapters below state that rule, derive its consequences, show which machines carry it, and prove where a window that hides the record and the ledger loses information.

Chapter 3 states the model, its toll, the member predicate, the smallest member, the history lift and the universal floor. Chapter 4 derives the toll on finite machines from the price of merging states and checks which premises are needed. Chapter 5 states what a window can decide and price. Chapter 6 states the recursion and undecidability results, over any substrate and for L. Chapter 7 states the record axis over any base and the latch decomposition of growing records. Chapter 8 states the physical boundary. Chapter 9 states the models of external systems and the pointer thesis.

Chapter 2

Notation and Conventions

- $\mathbb{N}$ is the natural numbers, $\mathbb{B} = \{\text{false}, \text{true}\}$, $\mathbb{R}$ the reals.
- A *trace* is a finite list of instructions. $\text{run}(\tau, s)$ folds the step function over τ from s.
- $(R, \leq)$ is the preorder of records and $\text{rec} : S \to R$ the record. In Coq it is `ax_rec` on an `AxSys`.
- P is the record on the two-point order, a reading from states to $\mathbb{B}$. In Coq it is `cs_cert` on a `CertificationSystem`.
- $c(i) \in \mathbb{N}$ is the cost of instruction i; $c(\tau)$ is the sum over a trace.
- μ is the ledger. Where a state carries it, conservation means $\mu(\text{step}(s, i)) = \mu(s) + c(i)$.
- A *window* is a function from states to observations.

Chapter 3

Thiele Machines and the Universal Floor

Definition 3.1 (Priced record system; `AxSys` in Coq)**.** A priced record system is a tuple $(S, I, \mathrm{step}, c, \mathrm{rec})$ with $\mathrm{step} : S \times I \to S$, $c : I \to \mathbb{N}$ and $\mathrm{rec} : S \to R$, where $(R, \leq)$ is a preorder given by a Boolean test. The record is a position in that order. A step from s by i *leaves the down-set* when $\mathrm{rec}(\mathrm{step}(s,i)) \not\leq \mathrm{rec}(s)$. The one rule is the toll, first named No Free Insight:

$$\textbf{A2:} \quad \forall s\, i,\ \mathrm{rec}(\mathrm{step}(s,i)) \not\leq \mathrm{rec}(s) \implies c(i) \geq 1.$$

Coq: the record `AxSys` in `coq/kernel/foundation/AxCore.v`, with the toll as `ax_a2`. The order carries no further assumption: not antisymmetry, not growth of the record along steps, not finiteness. An internal ledger is not part of this definition or the next; the results that use one say so.

Definition 3.2 (Thiele machine; `ax_thiele` in Coq)**.** A Thiele machine is a priced record system together with a set of designated starts $\mathrm{starts} \subseteq S$ such that

- (toll) A2 holds;
- (permanence) $\forall s\, i,\ \mathrm{rec}(s) \leq \mathrm{rec}(\mathrm{step}(s,i))$;
- (distinction) for some $s_0 \in \mathrm{starts}$ and some trace τ, $\mathrm{rec}(\mathrm{run}(\tau, s_0)) \not\leq \mathrm{rec}(s_0)$.

A reading is *flat* when $\mathrm{rec}(\mathrm{step}(s,i)) \leq \mathrm{rec}(s)$ for every s and i. Flat is a property of the reading. In the other direction, a system with a Boolean equality on its states and a run that ends off its start is what some Thiele machine looks like through a window that drops the record (Theorem 3.2). Coq: `ax_thiele` in `coq/kernel/foundation/AxThiele.v`, with the clauses `ax_a2`, `ax_grows` and `ax_distinction`, and `ax_flat` for the flat reading. Every Thiele machine has a move that costs at least 1 (`thiele_has_a_priced_move`) and has the floor over its order of Theorem 3.3 (`thiele_floor`).

Theorem 3.1 (The smallest Thiele machine)**.** ***(S)*** *The latch has two states* no $<$ yes*, one move that sends both to* yes *at price* 1*, and the state as its reading. It is a Thiele machine with start* no *(`latch_is_thiele`), and its move is not injective: both states land on one (`latch_merges`). Every Thiele machine has two states with different readings (`no_one_state_member`); over the two-point order every Thiele machine has a move that raises the reading from some no-state and keeps every yes-state at yes (`two_point_member_raising_move`); on two states read as themselves every Thiele machine has the latch's move (`two_state_member_has_latch_move`); and the toggle, whose move swaps the two states, is not a Thiele machine (`toggle_is_not_thiele`).*

Theorem 3.2 (The history lift)**.** ***(S)*** *Let M be any tuple $(S, I, \mathrm{step}, c, \mathrm{rec})$ of that shape, toll or not, with a Boolean equality test on its states that agrees with equality, and x a state. The history lift of M at x has states $S \times \mathbb{B}$, the moves of M, step $((s,b),i) \mapsto (\mathrm{step}(s,i),\ b \vee [\mathrm{step}(s,i) = x])$, price 1 on every move, and reading $(s,b) \mapsto b$ (`history_lift`). Dropping the bit returns the run of M (`history_lift_shadow`). If some run from a state $s_0 \neq x$ ends at x, the lift is a Thiele machine with start (s_0, false) (`history_lift_is_thiele`); so every M with a run that ends on a state other than its start is the shadow of a Thiele machine (`moving_machine_is_a_shadow`). Above every shadow sit two lifted states with different readings, and no function of the shadow equals the reading (`shadow_hides_history`). Over a finite base the least price list that meets the toll for the lift is 1 on every move that can land on x and 0 elsewhere, the toll holds for a price list exactly when it is at least that list, and the lift priced by that list is a Thiele machine (`lift_can_raise_iff`, `lift_toll_iff_least`, `least_lift_is_thiele`, `coq/audit/AxThieleLeast.v`).*

The flat reading, and computation as a shadow. A reading is flat when no step leaves its down-set, so A2 holds with nothing to charge (`flat_obeys_toll`), no run of it leaves the down-set of its start (`flat_has_no_distinction`), and it is not a Thiele machine for any choice of starts (`flat_is_not_thiele`). A clock and a reversible process, each read through a reading its moves never raise, are flat, and so is any deterministic transition system (S, I, step) given a constant record: forget c and rec from any Thiele machine and what remains is flat. Flat is a property of the reading, not of the machine: every system with a Boolean equality test on its states and a run that ends on a state other than its start is the shadow of a Thiele machine (Theorem 3.2). Ordinary computation is one flat shadow among others, and the theorems of this reference are about what the forgetting throws away.

The one-bit instance. Take $R = \mathbb{B}$ with false $<$ true (`two_pre`). The record is a reading $P : S \to \mathbb{B}$, leaving the down-set is a false-to-true crossing, and A2 reads

$$\forall s\, i,\ P(s) = \mathrm{false} \wedge P(\mathrm{step}(s,i)) = \mathrm{true} \implies c(i) \geq 1.$$

Coq: the record `CertificationSystem`, with this instance of A2 as its field `cs_cert_costs`; `cs_axsys` and `axsys_cs` translate each way. The name of that record does not restrict the interpretation of P: it is an arbitrary Boolean reading, and neither permanence nor the occurrence of both values is required. Most results below are stated on this instance, and each says so by writing P. A certification system obeys the toll on this instance; it is a Thiele machine only with permanence and a distinction added.

Theorem 3.3 (The floor is the toll)**.** ***(S)*** *For every priced record system, every run whose final record is not at or below its starting record costs at least 1 exactly when A2 holds (`ax_floor_iff_a2`), and under A2 every run costs at least the number of its steps that leave the down-set (`ax_cost_ge_exits`). When the record never drops along a step, leaving the down-set is a strict rise (`ax_exit_is_strict_rise`).*

Proof. A one-step run is a step, so the floor gives the toll. For the converse, induct on the run. If the first step stays in the down-set, the rest of the run ends outside the down-set of its own start, by transitivity of the order, and the induction hypothesis pays. If the first step leaves the down-set, A2 pays. Counting exits is the same induction. □

Theorem 3.4 (Thresholds are one-bit machines)**.** ***(S)*** *For every point a of R, reading the record only as the bit "$a \leq \mathrm{rec}(s)$" gives a one-bit priced record system, and it obeys its toll whenever the original does (`ax_view_a2`); the original obeys A2 exactly when every such threshold does (`ax_a2_iff_views`). When the order is antisymmetric, the thresholds determine the record (`bp_thresholds_determine`). The one-bit floor of Theorem 3.9 is the two-point instance (`two_point_nfi`).*

Sliced interface. On the one-bit instance, a sliced system additionally has a surjection $\pi : S \to B$ and a partition $I = I_H \sqcup I_V$. Write $F_i(s) = \mathrm{step}(s, i)$. For every $i \in I_H$ there is $h_i : B \to B$ with

$$\pi(F_i(s)) = h_i(\pi(s)), \qquad P(F_i(s)) = P(s), \qquad c(i) = 0.$$

For every $v \in I_V$, require $\pi(F_v(s)) = \pi(s)$ and $P(s) \leq P(F_v(s))$, ordering false below true. A2 still applies. These equations hold on all states of the stated domain; they imply permanence. They do not require a product state space or a universal base.

For a non-certification instance, use $S = \mathbb{Z}_4 \times \{0, 1, 2\}$, $\pi(b, k) = b$ and $P(b, k) = [k = 2]$. The horizontal instruction $R(b, k) = (b + 1 \bmod 4, k)$ costs zero; the vertical instruction $V(b, k) = (b, \min(k+1, 2))$ costs one. Projection and reading preservation for R are immediate; V preserves the projection, never lowers the reading, and pays one on its crossing from $k = 1$ to $k = 2$. It also costs one at $k = 2$.

Valuations. A Boolean valuation need not itself be an autonomous two-state quotient: `coq/audit/ValuationBoundary.v` gives a counterexample.

3.1 Free moves, least prices and observation

Theorem 3.5 (All readings and free moves)**.** ***(S)*** *For a state type with decidable equality, fixed transitions and natural instruction prices, A2 holds for every Boolean reading if and only if every zero-cost instruction is the identity on all states. Coq:* `every_reading_toll_iff_free_moves_are_identity`*.*

Proof. If a free move changes s to t, the decidable reading $[x = t]$ is false at s and true at t, contradicting A2. Conversely an identity move cannot raise any reading, and every positive natural price is at least one. □

This distinguishes a chosen priced reading from arbitrary state predicates; it does not select an autonomous base projection or a partition of moves.

Theorem 3.6 (Free reading preservation)**.** ***(S)*** *Under A2 and global permanence, every zero-cost instruction preserves the reading. With the other sliced-interface laws fixed, global permanence can replace horizontal reading preservation and vertical monotonicity. Coq:* `free_reading_invariance`*.*

Proof. A free move cannot raise the reading by A2 and cannot lower it by permanence. Conversely, horizontal preservation and vertical monotonicity ensure that no instruction lowers the reading, since the two classes exhaust the instruction set. □

The existing `SlicedInterface` uses the latter presentation; its derived permanence is not an independent premise for removing a field.

Theorem 3.7 (Least instruction toll)**.** ***(S)*** *Fix the transition and reading, and supply a finite list covering all states. For each instruction i, set $p(i) = 1$ if some state has a false-to-true crossing under i, and $p(i) = 0$ otherwise. Then A2 holds for a natural schedule c exactly when $p(i) \leq c(i)$ for every i. The same equivalence holds for the global finite-trace crossing floor. Consequently p is the unique pointwise least valid schedule. Coq:* `toll_exactly_dominates_least_price`*;* `trace_floor_exactly_dominates_least_price`*;* `unique_least_valid_price`*.*

Proof. Search the covering list. A crossing witness forces every valid instruction price to be at least one. Without a witness, zero meets every demand for that instruction. The resulting schedule is valid and no larger than any valid schedule. Antisymmetry proves uniqueness. A2 implies the trace floor by the first crossing, while one-step traces give its converse. □

The finite list supplies an effective search; it is not a minimization algorithm for arbitrary infinite systems.

Theorem 3.8 (The coarsest observational identification)**.** ***(S)*** *For a total deterministic system with specified observation o, let $s \sim t$ mean $o(\mathrm{run}(w, s)) = o(\mathrm{run}(w, t))$ for every finite word w, including the empty word. This is the unique coarsest observation-preserving, step-stable equivalence. Any deterministic implementation preserving steps and observations must separate states that some future observation separates. The coarsest relations are pointwise equivalent. Coq:* `future_equivalence_coarsest`*,* `coarsest_relation_unique`*,* `encoding_cannot_merge_distinguishable_states`*.*

Proof. Equality for each word gives the equivalence laws. The empty word gives current-output preservation; prefixing a move gives step stability. Any stable output-preserving relation preserves outputs along a word by induction, so is contained in $\sim$. An implementation merging a separated pair would have to give two answers after the same continuation from one state. □

Uniqueness here is pointwise logical equivalence of relations.

Histories and maps. Give a pointed source machine an access word reaching each state. A start- and step-preserving map into a pointed target exists exactly when equal source endpoints of any two words imply equal target endpoints. Necessity follows by mapping the endpoints. For sufficiency, map a source state to the target endpoint of its access word; the implication makes this independent of all other words reaching it. The empty word and appending a move give the required laws, and reachability forces pointwise uniqueness. Coq: `unique_factorization_iff_kernel_inclusion`.

If two reachable machines have the same initial output behavior and each is separated by future observations, this implication follows by appending all continuations and using target separation. With supplied access words for the first machine, the resulting map preserves outputs and is onto by target reachability and injective by source separation. It is pointwise unique (`minimal_realizations_uniquely_isomorphic`). This is behavioral minimality, not a claim about smallest infinite cardinality.

Cost interfaces. The definition uses instruction costs $c : I \to \mathbb{N}$, so a trace's cost is independent of its initial state. Later source-charge results use a broader interface $a : S \to \mathbb{N}$ for one autonomous step, or $\widetilde{c} : S \times I \to \mathbb{N}$ with instructions. Their trace cost is the sum of charges along the actual run. The same first-crossing proof gives a floor if each crossing costs at least one, but instruction-priced theorems retain their stated signature and hypotheses. In particular, a single instruction with one fixed price cannot charge one at entry and zero at idle. The latter construction is the source-charge example `paid_entry_free_idle` in `minimal/AccountingBridge.v`.

Theorem 3.9 (The universal floor)**.** ***(S)*** *In every one-bit system that obeys the toll, every one-bit Thiele machine among them, if $P(s_0) =$ false and $P(\mathrm{run}(\tau, s_0)) =$ true, then $c(\tau) \geq 1$. Coq:* `universal_nfi_any_substrate`*; it is also the two-point instance of the floor over any order (*`two_point_nfi`*).*

Proof. Induction on τ. Since P is false at the start and true at the end, some step in τ takes it from false to true. By A2 that step costs at least 1; every other step costs at least 0. □

Theorem 3.10 (Simulating systems)**.** ***(S)*** *Let a certification system, the base, be simulated by another, the host: an embedding of base states into host states, a decoding of each base instruction into a list of host instructions, a commuting step law, and certification reflected by the embedding (record* `SimulatingCertificationSystem`*). A base trace from uncertified to certified costs at*

least 1 *in the base, the host run of its decoded trace ends certified, and that host run costs at least* 1*. Coq:* `host_represents_simulating_cert_system`*.*

The commuting law compares endpoints of complete translated instruction blocks. The host's cost floor uses its own toll axiom. This theorem alone neither constrains intermediate block states nor establishes an earned history for arbitrary host programs. Such guarantees require preservation over the host's permitted operations and initial states. The checked counterexample in `coq/audit/MacroBoundary.v` separates these claims.

Theorem 3.11 (Quantitative form)**.** ***(S)*** *Extend a certification system with a witness $w : S \to \mathbb{N}$ and a threshold k such that each step raises w by at most its cost and every certified state has $w \geq k$ (record* `QuantitativeCertificationSystem`*). A trace from a state with $w = 0$ to a certified state costs at least k. Coq:* `universal_nfi_quantitative`*.*

Theorem 3.12 (A2 is a restriction)**.** ***(S)*** *A cost-bearing system without A2 can switch certification on at zero cost, and A2 excludes exactly that. Coq:* `honest_cost_tracking_strict_restriction`*,* `free_forgery_violates_A2`*.*

3.2 A2 among cost frameworks

Theorem 3.13 (The toll as amortized cost)**.** ***(S)*** *A step system satisfies A2 if and only if every step has non-negative amortized cost under the potential that is* 1 *on uncertified states and* 0 *on certified ones; the universal floor is then the potential method's telescoping bound. Coq:* `a2_iff_nonnegative_amortized_cost`*,* `nfi_by_potential`*. Under A2 the number of certification flips along a trace is at most its cost (*`flips_le_cost`*), and A2 together with the matching upper-bound condition is equivalent to exact pricing of a permanent certificate (*`a2_and_aara_iff_exact`*).*

These are local correspondences. The cost-framework families they touch (graded monads, amortized resource analysis, cost translations, linear logic) are not embedded here.

3.3 Several floors and one charge

Theorem 3.14 (Joint floors)**.** ***(S)*** *For a finite family of events, the least cost schedule meeting every unit floor charges one on a step where any event occurs and zero otherwise. Coq:* `joint_floor_is_least`*. On pairs of Booleans, two independent coordinates that both rise in one step are met by a single charge of* 1 *(*`independent_coordinates_joint_change_costs_one`*).*

Independent state coordinates do not by themselves force additive cost.

Theorem 3.15 (Exact pricing)**.** ***(S)*** *For trusted local-predicate pricing systems, a schedule meets the certification floor and never overcharges exactly when it charges on precisely the certification flips, one unit each, and nothing elsewhere. Coq:* `exact_commitment_pricing_characterization`*.*

Chapter 4

Finite Machines and the Price of Merging

Fix a state type S with a finite enumeration, instructions I, a step function and a reading P. The certificate is *permanent* when no step switches it off. An instruction is *injective* when its step function is.

Theorem 4.1 (A permanent flip merges)**.** ***(S)*** *On a finite state space with a permanent certificate, an instruction that switches the certificate on at some state is not injective. Coq:* `permanent_flip_is_not_injective`*.*

Proof. If the instruction were injective it would map the certified states injectively into the certified states, so onto them, leaving no certified image for the uncertified state it switches on. □

This asserts non-injectivity of the instruction, not a collision at the entry destination. For $u \notin H$, $f(u) \in H$, and $f(H) \subseteq H$ with $|H| = K$, the path $u \to h_1 \to \cdots \to h_K \to h_K$ delays the orbit's collision until the $(K+1)$st transition. The entry shares its destination with a source in H exactly when $f(u) \in f(H)$. Surjectivity of $f|_H$ is sufficient, not necessary, for this condition at a particular entry. Instruction pricing charges every use of a non-injective instruction under the merge-pricing premise, regardless of where its collision occurs.

Theorem 4.2 (A2 from merge pricing)**.** ***(S)*** *On a finite state space with a permanent certificate, if every non-injective instruction costs at least* 1*, A2 holds. Coq:* `a2_from_merging_price_and_permanence`*; the trace form is* `permanent_certification_trace_floor`*, and an honest erasure account gives A2 too (*`honest_erasure_accounting_implies_a2`*).*

Each premise is needed. Unbounded history walks past finiteness by paying in fresh states, a new one every step (`unbounded_history_escapes`); a revocable certificate keeps nothing (`revocable_certificate_escapes`); and an unpriced merge is a price list that skips the merge (`free_merge_escapes`). On any state space, a raise that merges nothing sends the certified states one-to-one into a proper part of themselves, so it needs infinitely many of them.

Theorem 4.3 (The squeeze bound)**.** ***(S)*** *On a finite state space with a permanent certificate, with m certified states and k uncertified states that one instruction switches on, the instruction collapses at least k states (*`permanent_flips_collapse_at_least`*). Priced one unit per halving of the number of distinct states, its cost c satisfies $m + k \le 2^c m$, so, when $m \ge 1$, $c \ge \log_2((m+k)/m)$; in rounded-up logarithms, $\lceil \log_2(m+k) \rceil \le c + \lceil \log_2 m \rceil$. Coq:* `permanent_flips_compression_bound`*,* `permanent_flips_log_bound`*.*

These bounds are tight, and A2 is equivalent to the floor. For every system, A2 holds exactly when every run from an uncertified state to a certified one costs at least 1 (`nec_f_floor_iff_a2`). For every $m \geq 1$ and c, a grid machine with m certified states and one move of cost c flips $2^c m - m$ states, so the bound $m + k \leq 2^c m$ of the theorem above is met with equality (`nec_f_squeeze_tight`). Its rounded-logarithm form is strictly weaker: $m = 3$, $k = 1$ and cost 0 meet the rounded form and fail $m + k \leq 2^c m$ (`nec_f_rounded_log_form_weaker`).

Theorem 4.4 (Which flips are forced)**.** ***(S)*** *When instructions have decidable equality, over all schedules that price merges, an instruction is forced to have positive price if and only if it merges two states (`forced_priced_iff_merges`). An instruction that switches a reading on either merges or switches it off somewhere else (`flip_merges_or_revokes`). So every permanent-record write is forced, and the only flips a merge-priced schedule can leave unpriced are revocable ones, which keep nothing.*

Theorem 4.5 (Refutation)**.** ***(R)*** *"The forced events are exactly the permanent-record writes" is false: a three-state instruction flips, revokes, and merges at once. Coq: `forced_price_without_permanent_record`.*

Theorem 4.6 (Merge, or a fresh state every step: the dichotomy)**.** ***(S)*** *On any state space, let a reading be permanent under a step f, and take a run $x_0, f(x_0), f(f(x_0)), \ldots$ in which the reading goes from false at step r to true at step $r + 1$. If no two states among the run's first n steps merge under f, those states are pairwise distinct; so on a finite state space of M elements, a run that keeps a record and never merges has fewer than M steps. In particular, for an injective move F that switches the reading on at s, the iterates of s are pairwise distinct and no finite duplicate-free list contains them all: a permanent flip either merges or visits a fresh state every time. Coq: `run_merges_or_fresh`, `run_bounded_by_room`; `permanent_flip_iterates_distinct`, `permanent_flip_no_finite_list`.*

Theorem 4.7 (Retention bound, and its exactness)**.** ***(S)*** *Let a finite implementation $F : X \to X$ of a reset carry an abstraction q that sends every image of F to a fixed abstract state q_0, with a representative of q_0 and representatives of $N - 1$ further, pairwise distinct abstract states, and let F be injective on the states reached from the representatives in fewer than T applications. Then $|X| \geq 1 + (T + 1)(N - 1)$; for $N \geq 2$, the history is therefore kept through at most $\lfloor (|X| - N)/(N - 1) \rfloor$ resets. For $N = 1$ the bound is only $|X| \geq 1$ and gives no finite retention horizon. The bound is attained: one fixed state and $N - 1$ paths of $T + 1$ states, with F walking each path and sending its last state to the fixed one, has exactly $1 + (T + 1)(N - 1)$ states, is closed under F, meets the reset and representative premises, is injective through T repetitions, and, when $N \geq 2$, merges on the next. Coq: `retention_bound`, `retention_horizon`; `tight_exact`, `tight_closed`, `tight_retain`, `tight_merges_next`.*

Theorem 4.8 (The step law)**.** ***(S)*** *For any step f on any type with decidable equality and any start, every step of the run before its first revisit is fresh (the new state equals no earlier state of the run); the first revisit lands on the start or is a merge (it equals an earlier state x_j with $j \geq 1$ and the state before x_j differs from the current state, so two distinct states share an f-image); and every later step repeats a step the run already took, so the run is periodic and merges nothing new. Coq: `step_law`, `first_revisit_start_or_merge`, `later_steps_repeat`, `merge_is_collision`, `revisit_means_loop`.*

Theorem 4.9 (Refutation: the halting hole)**.** ***(R)*** *"Every raise of a permanent reading on finitely many states is a merge" is false for partial steps: on two states, $a \mapsto b$ with b having no successor raises the reading with no shared image. Totalising the halt ($b \mapsto b$) makes the raise a merge. Coq: `partial_raises_without_merge`, `totalised_halt_is_merge`.*

Theorem 4.10 (The write of a cell, and the fresh states it takes to avoid the merge)**.** ***(S)*** *On the states blank, zero, one with reading "definite," the write of a 0 is permanent,*

raises the reading from blank, and merges blank with zero. A fourth state remembering that the cell was blank does not avoid the merge: the write on four states still merges two distinct states. Only endlessly many fresh states avoid it, a new one every step: on the natural numbers the write "add one" is injective and never returns to 0; on the integers "add one" is a bijection, raises "at least one" from 0, keeps it, and after $k > 0$ steps is away from its start. Coq: `write_is_permanent`, `write_raises`, `write_is_a_merge`, `four_state_escape_fails`, `tape_write_injective`, `tape_write_never_returns`, `shift_bijective`, `shift_raises_and_keeps`, `shift_never_returns`.

Theorem 4.11 (No state space codes all its own readings; the regress has a price)**.** ***(S)*** *For any type X and any coding $X \to (X \to \text{bool})$, some reading $X \to \text{bool}$ is coded by no state (Cantor). Under the halving price, a run that narrows N candidates to one costs at least $\lceil \log_2 N \rceil$ units, and for every m there is a candidate count whose rounded-up logarithm is at least m. Coq:* `no_space_holds_its_own_readings`, `regress_price_unbounded`; *the count itself is* `run_narrowing_priced_log`.

Theorem 4.12 (Nondeterministic merge)**.** ***(S)*** *Let a step relation on a finite duplicate-free list of states have a duplicate-free set Y of yes-states that is closed and total (every yes-state has a chosen successor in Y), and let some state outside Y relate to a state of Y. Then some state of Y has two distinct predecessors. Coq:* `nondet_merge`.

Theorem 4.13 (Reversible keeps nothing)**.** ***(S)*** *For an injective step on a finite duplicate-free list containing every state, the run from any start returns to the start after a positive number of steps, and any reading that is false at the start and never goes from true to false is false at every step. Coq:* `returns_to_start`, `reversible_keeps_nothing`.

Theorem 4.14 (Noisy steps, both directions)**.** ***(S)*** *Let P be a nonnegative kernel on n states with rows summing to 1 and let Y be closed under P. If every column of P sums to 1, then $P(i,j) = 0$ for every $i \notin Y$, $j \in Y$. If some $i \notin Y$, $j \in Y$ has $P(i,j) > 0$, then some column sum differs from 1, and the scaled collision weight $\sum_j c_j^2$, with c_j the column sums, strictly exceeds n. For the uniform distribution $u_i = 1/n$, $c_j = n(uP)_j$, so the actual collision probability $\sum_j (uP)_j^2$ exceeds $1/n$, its initial value. Conversely, if every column sums to 1, then for every spread q, $\sum_j (qP)_j^2 \le \sum_i q_i^2$. Over the reals, for a strictly positive spread q on N states, $\sum_i q_i \ln(Nq_i) = \ln N - H(q) \ge 0$, with strict inequality as soon as some $q_i \neq 1/N$. Coq:* `noisy_chairs`, `raise_breaks_uniform`, `colsums_total`, `raise_concentrates`, `uniform_floor`, `raise_concentrates_uniform`, `no_merge_no_concentration`; `gibbs_nonneg`, `gibbs_strict`, `relent_is_entropy_gap` *(the last three over Coq's real numbers).*

Theorem 4.15 (Unitary steps)**.** ***(S)*** *Over the algebraic closure of $\mathbb{Q}$ with complex conjugation, let U be an $n \times n$ matrix with $U^\dagger U = 1$ and P a matrix with $P^2 = P$, $P^\dagger = P$ and $(1-P)\,U\,P = 0$. Then $P\,U\,(1-P) = 0$. Stated over the Mathematical Components library; finite dimension enters once, as "a left inverse of a square matrix is a right inverse." Coq:* `quantum_chairs`.

Theorem 4.16 (Heat against time)**.** ***(S)*** *For $0 < p < 1$ the biased walk's entropy production per tick, $\sigma(p) = (2p-1)\ln\frac{p}{1-p}$, is 0 at $p = \frac{1}{2}$ and strictly positive otherwise. For $0 < v < 1$ and $N \ge 0$, $N \ln\frac{1+v}{1-v} \cdot \frac{N}{v} \ge N^2$. Over Coq's real numbers. Coq:* `heat_at_any_speed`, `no_heat_at_rest`, `heat_time_floor`.

The displayed constant is deliberately weaker than the best bound for this formula. Indeed, the derivative of $\ln((1+v)/(1-v)) - 2v$ is $2v^2/(1-v^2) > 0$ and the expression is zero at $v = 0$. Thus the product is at least $2N^2$, strictly so for $N > 0$. The cited Coq theorem asserts the stated N^2 bound.

Remark 4.1. The monograph also develops four further arguments: one unit per crossing (a run costs at least its number of no-to-yes steps, with no permanence assumed, and that bound

characterizes the toll; Coq: `nec_f_cost_ge_rises`, `nec_f_rises_iff_a2`); the certainty bound (under entropy pricing, the uncertainty about the start equals the part the run destroyed, at most its cost in bits, plus the part the end state still holds); every scale paying the same way (if a coarse description of a machine merges, the fine step merges two described states, leaves the described set, or the described set is infinite); and the device floor (under the stated thermal entropy-balance assumptions, a fixed protocol merging K inputs produces at least $k_B \ln K$ of entropy, at least $k_B T \ln K$ of dissipated work, on its worst input, however slowly it runs and whatever anyone knows).

4.1 Entropy and heat

Theorem 4.17 (Entropy after a permanent step)**.** ***(S)*** *After a permanent step, any distribution on the m certified states and the k flipping ones has Shannon entropy at most $\log_2 m$ (`permanent_step_entropy_ceiling`). From the uniform distribution the step removes at least $\log_2((m+k)/m)$ bits (`permanent_flip_uniform_entropy_drop`). An injective step removes none (`step_entropy_invariant_if_injective`).*

Theorem 4.18 (Heat)**.** ***(C)*** *Given the named premise `landauer_heat`, which states Landauer's principle for a step and a distribution (Rolf Landauer argued at IBM in 1961 that a step that cannot be run backwards requires a minimal heat generation; the premise reads this as mean heat at least $k_B T \ln 2$ per bit of entropy the step removes, for the chosen physical memory and its input distribution, as an average over that preparation rather than a bound on each trial), the instruction has mean heat, when $k_B T \geq 0$, at least $k_B T \ln((m+k)/m)$ from the uniform distribution (`permanent_flip_heat_floor`), and, when $k_B T > 0$, positive heat from any distribution that leaves every certified state and the flipping one possible (`permanent_flip_full_support_heat_positive`). For a state known in advance, the premise is equivalent to heat ≥ 0, so it supplies no positive lower bound; it does not determine actual device heat, which depends on the physical process and preparation, and the ledger toll still charges its unit on a known run, since merge pricing prices the instruction whatever input was prepared (`known_state_flip_forces_no_heat`).*

The monograph adds, on the page and with no Coq counterpart here, a floor that comes from the device rather than the premise. By the identity tying Ladyman's reading of Landauer's principle to Sagawa's, and the second law, a fixed device that merges K inputs produces at least $k_B \ln K$ of entropy, at least $k_B T \ln K$ of dissipated work, on its worst input, however slowly it runs and whatever anyone knows. So on a known start the premise sets no floor, and the merging move still has one on its worst input.

4.2 An eight-state instance

Example 4.1. Four program slots and a certification flag make eight states. An advance step is injective (`fnext_injective`); the certify step and every jump merge states (`fcertify_merges`, `fjump_merges`). The cost rule prices exactly the merges, so A2 holds there as a theorem (`fin_a2_from_merging_price`).

4.3 No intrinsic joule value

Theorem 4.19 (Refutation)**.** ***(R)*** *For every state type and natural-number ledger, the energy assignments at scales 1 and 2 give values 1 and 2 respectively at every state whose ledger is 1. This is conditional on such a state; an identically zero ledger need not yield different energies. The ledger fixes no energy unit. Coq: `mu_has_no_intrinsic_joule_value`. Choosing the scale*

$k_B T \ln 2$ gives one ledger unit that energy by definition (`calibrated_mu_landauer_energy`); the scale is a premise, and nothing here measures it.

Chapter 5

Windows, Pricing and Narrowing

Theorem 5.1 (Window adequacy)**.** ***(S)*** *A query on states can be answered by a function of a window only if it is constant on the window's fibers, the sets of states the window shows the same way (`decoding_requires_fiber_constancy`). Given a representative for every view, fiber constancy is equivalent to factoring through a decoder (`selected_representatives_give_decoder`).*

Adequacy for a supplied operational model. Allow a relation of permitted successors, a chosen state view and required outputs. Supply a representative for each view and a permitted successor for each instruction and underlying state. A deterministic decoder and next-view rule exist exactly when equal views give equal current outputs and every permitted successor under a common instruction has the same view from equal-view sources. This is `operational_adequacy_iff`; it does not establish that the supplied relation describes a physical apparatus, or infer effective choices from existential ones. Observable branching from the same state and instruction excludes an exact deterministic encoding preserving those outcomes (`branching_answers_exclude_exact_deterministic_encoding`).

Selection and finite validation. For the unpointed two-state toggle, every Boolean reading invariant under its state-exchange symmetry is constant (`toggle_symmetry_forces_constant_reading`). Distinguished starts or task meanings are additional structure. For every finite horizon a unit-priced clock with a delayed reset agrees through that horizon with a permanent reading, while both satisfy the actual toll and only one is permanent (`finite_observations_do_not_establish_permanence`). These results constrain conclusions from bare dynamics and bounded tests; they neither refute every physical permanence model nor choose a universal physical description. Sources: `coq/audit/ObservationContract.v` and `coq/audit/IntrinsicBoundary.v`.

Theorem 5.2 (Shadow pricing)**.** ***(S)*** *If a certifying step and a non-certifying step show the same window before and after, no price computed from the window meets the certification floor without charging the non-certifying step. Coq: `shadow_cannot_price_exactly`. A window that shows the reading prices certification exactly (`window_showing_reading_prices_exactly`).*

Theorem 5.3 (Retained history)**.** ***(S)*** *Keeping the previous state in a history makes every deterministic step injective, and the state part of the new pair is still the original step's result, so the observed transition is unchanged. The history takes a fresh state every step; it walks past the finiteness premise, not past what keeping takes. Coq: `retained_history_step_injective`, `history_simulates_observed_step`.*

Theorem 5.4 (Narrowing)**.** ***(S)*** *On a finite machine priced one unit per halving of the number of distinct states, each halving of the machine's own set of possible states along a run is paid for (`run_narrowing_priced_log`). What an observer knows can shrink at zero*

cost: a measurement that copies a hidden bit into a blank display forgets nothing, borrowing the blank display (fresh states), and one admissible merge-priced cost assigns it zero (`observer_narrowing_can_be_free`); wiping the display is a merge and costs at least one (`wipe_costs_at_least_one`). The WIPE charge is an instruction-level bound over all its possible sources. On the particular correlated pair produced by copying, the hidden bit can retain the entire distinction after WIPE. A charge of one is therefore not a claim that this preparation loses one bit of joint entropy, or dissipates one corresponding heat unit.

Chapter 6

Recursion and Undecidability

Definition 6.1 (Substrate)**.** A substrate supplies states, programs, a partial run function, a ledger that no step decreases, an encoding of programs into states with a decoder, behavioral equivalence (equal runs from every state), a representability predicate on program transformers, and the recursion theorem stated and proved by the logician Stephen Kleene in 1938, for representable transformers, as a field (class `Substrate`). A shortcut predicate on programs respects behavioral equivalence and has one accepted and one rejected canonical program (class `WithShortcutPredicate`).

Theorem 6.1 (Structural undecidability)**.** ***(S)*** *In every substrate with a shortcut predicate, no decider whose flip transformer is representable returns true exactly when the predicate holds. Coq:* `structural_shortcut_undecidable`*.*

Proof. Let D be such a decider and $f(p)$ the rejected canonical program if $D(p)$ is true and the accepted one otherwise. The recursion theorem gives q behaviorally equivalent to $f(q)$. If $D(q)$ is true, q behaves like the rejected program, so the predicate fails at q; if false, it holds. Either way D is wrong at q. □

The natural-number substrate meets the recursion premise by construction, since its fixed-point program copies the transformer's output (`nat_structural_shortcut_undecidable`).

Theorem 6.2 (L)**.** ***(S)*** *For the weak call-by-value lambda calculus L, defined in* `coq/kernel/foundation/LRecursion.v` *from its reduction rules, Kleene's second recursion theorem holds (*`second_recursion`*), every extensional nontrivial property of L terms is undecidable (*`L_rice`*), and L halting is undecidable (*`L_halting_undecidable`*). L is the calculus Yannick Forster and Gert Smolka presented as a model of computation in which computability theory can be carried out in Coq (Interactive Theorem Proving, ITP 2017, LNCS 10499). Yannick Forster, Fabian Kunze, Gert Smolka and Maximilian Wuttke later checked in Coq that L and Turing machines simulate each other (Interactive Theorem Proving, ITP 2021). The Turing-completeness of L is theirs and is not proved in this repository.*

The diagonal uses recursion and behavioral equivalence; cost plays no part. Deciding a predicate over arbitrary programs is a different task from reading one field of a given state.

Chapter 7

The Record Axis

Definition 7.1 (Bases and base-driven records)**.** A base is a deterministic record-free machine: states, a next function, initial states and halting (record `BaseMachine`). A record-carrying machine has states, a next function, initial states, a record, a ledger and halting (record `RCM`). A cover maps it onto a base step for step, preserving initial states and halting. The extension is *base-driven* when its record's next value depends only on the base state and itself, the ledger never decreases, A2 holds, the record is permanent, and some reachable step writes it (`HonestBaseExtension`).

Theorem 7.1 (The record axis is a latch)**.** ***(S)*** *Every base-driven record over every base factors as a latch: there is a base event h such that the base state and the record evolve as $(b, r) \mapsto (\mathrm{next}(b), r \vee h(b))$. Coq: `record_axis_is_latch_holds`. Two permanent records factor as two latches, each reading the other (`record_pair_is_two_latches_holds`).*

Theorem 7.2 (Both premises do work)**.** ***(R)*** *A record that can switch back off is not a latch (`toggle_not_latch`). A record switched on by a hidden clock is permanent and not driven by the computation (`clock_record_permanent`, `clock_record_not_driven`).*

Theorem 7.3 (Which machines carry the axis)**.** ***(S)*** *Every base carries it: the base plus a latch on any event it reaches, charging one unit when the latch sets, is a base-driven record (`latch_core_honest`). A reversible base with unbounded memory carries it reversibly (`history_latch_honest`, `history_latch_injective`). With finite memory, a step that writes a permanent record is not injective (`finite_reversible_cannot_write`).*

7.1 Growing and probabilistic records

Theorem 7.4 (Growing records as families of latches)**.** ***(S)*** *A record whose next value is a function of the base state and its current value, that only grows in a partial order, whose ledger charges every strict change, and that changes at some reachable step satisfies a complete family of threshold-latch equations and keeps its price schedule (`growing_record_decomposes_holds`). A monotone chain of pairwise-distinct k-bit vectors has at most $k + 1$ members (`chain_needs_bits_holds`).*

Theorem 7.5 (Limits of one latch and a fixed schedule)**.** ***(R)*** *One Boolean latch does not carry every growing record: a three-value chain over a one-state base is a counterexample (`one_latch_refuted`). In a finite branching model a deterministic latch does not handle branching (`deterministic_latch_handles_branching_refuted`), and equal branch support and charges do not determine the branch weights (`schedule_determines_probabilities_refuted`).*

7.2 Bases

The relation `weak_base_equiv`, agreement on chosen observations and halting with stuttering steps, is an equivalence (`weak_base_equiv_refl_holds`, `weak_base_equiv_sym_holds`, `weak_base_equiv_trans_holds`). The record axis is a latch over the Cook and Reckhow RAM, the unit-cost random-access machine Stephen Cook and Robert Reckhow defined to measure time bounds (Journal of Computer and System Sciences 7, 1973), and over L (`record_axis_is_latch_on_ram_holds`, `record_axis_is_latch_on_l_holds`); these instances follow from the general theorem and do not distinguish the bases. Cost-respecting simulations between different bases are not constructed.

7.3 The record over any order

The definition of Chapter 3 has the record in any preorder, and the theorems there state its floor and its thresholds. In Coq the priced record system is an *axis system* (`AxSys`); the member predicate is `ax_thiele`. This section states the further results on that axis.

Theorem 7.6 (Latches, merges and windows on the axis)**.** ***(S)*** *A record that never drops and whose next value is a function of the base state and the current record factors through thresholds: some event h makes a at or below the next record exactly when a is at or below the current record or h fires (`ax_decompose`). A repeat-free chain of k-bit words, each word's set bits among the next word's, has at most $k+1$ members, and some chain has exactly $k+1$ (`chain_bits_exact`). With finitely many states and a growing record, every instruction that leaves the down-set from some state merges two states (`ax_growth_exit_merges`), and pricing every merge gives the toll (`ax_a2_from_merge_price`). For finitely many states and moves and decidable observations, some price computed from a window is exact exactly when the window has no collision (`ax_window_exact_iff_no_collision`).*

Theorem 7.7 (Entropy on the axis)**.** ***(S)*** *Under the halving price, on a state space of any size, for a non-negative weighting and any finite list of landing spots, each given with a repeat-free list of exactly the states the move sends to it, the move's conditional entropy loss over those lists is at most their total weight times the move's cost times* $\ln 2$ *(`ax_inf_loss_bits`).*

Chapter 8

The Physical Boundary

Theorem 8.1 (Calorimeter)**.** ***(S)*** *In a two-state calorimeter protocol, a canonical reset from half-excited to ground satisfies a discrete master equation exactly (*`canonical_reset_satisfies_master_equation`*), and at fixed Hamiltonian the bath receives* $\Delta/2$ *(*`canonical_reset_heat_exact`*).*

Theorem 8.2 (Refutation)**.** ***(R)*** *Population dynamics do not fix heat: two gaps with identical dynamics release different heat (*`master_equation_does_not_fix_heat_scale`*).*

Ruling a gap out needs a thermal premise such as local detailed balance, or device evidence. Converting the ledger to energy requires thermal and device premises; the ledger has no intrinsic energy unit (Chapter 4).

Chapter 9

External Systems and the Pointer Thesis

Each model below is narrow. None is a full security, consensus, database or audit-log theorem.

- **RFC 9162 (Certificate Transparency).** The iterative inclusion and consistency checks of the specification reject the boundary cases (`rfc9162_inclusion_boundary_safe`, `rfc9162_consistency_boundary_safe`) and accept the specification's examples under symbolic digests (`rfc9162_example_inclusion_d0`, `rfc9162_example_consistency_4_7`). Collision resistance and signed-tree-head authenticity are cryptographic premises; neither is proved.
- **Proof-carrying code.** A toy checker accepts exactly when the address-bound verification condition holds (`pcc_checker_accepts_iff_vc`). It mirrors the roles in George C. Necula's proof-carrying code (POPL 1997), where a program ships with a proof that it is safe; the full system is outside this model.
- **Trusted Platform Module (TPM) quotes.** A bare sign/verify interface does not imply quote authenticity (`tpm_interface_authenticity_refuted`).
- **Casper FFG.** Casper FFG is the finality gadget Vitalik Buterin and Virgil Griffith proposed for Ethereum in 2017. Two finalized blocks on conflicting branches imply a slashable quorum (`accountable_safety`). This is the accountable-safety theorem of their paper.

The addressed-RAM and reversible-update files prove only their named fragments. The cost-semantics correspondences are not full cost-framework embeddings.

9.1 The pointer criterion

The pointer-observable criterion proposes that the event a step rule is forced to price is singled out by redundant record proliferation across observers. Twelve candidate events over seven observer maps proliferate as stated (`twelve_candidate_measurements_checked`). The MAC-labelled model does not proliferate (`mac_model_not_proliferating`); applying that to a real system depends on the observer model. A two-observer toggle game with consensus, authenticity and coordinator-free updates revokes its event (`toggle_game_refutes_strong_pointer_necessity`), so those properties do not force permanence. Durable views do imply permanence, by making persistence a premise (`durable_consensus_implies_permanence`). The external consequence survey contains five narrow countermodels, each for a point a governing source already addresses, and five stronger

statements without enough protocol or hardware semantics; no surveyed consequence is simultaneously record-axis dependent, novel, and ready for use. The pointer criterion is a thesis: "faithfully model" and "deployed" tie it to the world and cannot be defined, as in the Church–Turing thesis.

Appendix A

Proof File Index

Results	File
The member predicate, the latch and the history lift	coq/kernel/foundation/AxThiele.v, coq/audit/AxThieleLeast.v
The definition, its floor and the one-bit instance	coq/kernel/foundation/AxCore.v, coq/kernel/nfi/UniversalCertificationCost.v
Valuations	coq/audit/ValuationBoundary.v
Free moves and least instruction toll	coq/audit/LeastToll.v
Future observations and unique realizations	coq/audit/CanonicalObservation.v, coq/audit/ObservationContract.v
Histories and minimality	coq/audit/TraceFactorization.v
Simulation boundary	coq/audit/MacroBoundary.v
Source charges and exact accounting	minimal/AccountingBridge.v
Fresh states and sharp retention	minimal/HeatOrRoom.v, minimal/SharpHost.v
First revisit, halting and kept distinctions	minimal/StepLaw.v, minimal/HaltingHole.v, minimal/KeptDistinction.v
Nondeterministic and reversible steps	minimal/NondetChairs.v, minimal/Recurrence.v
Stochastic concentration and entropy	minimal/NoisyChairs.v, minimal/GibbsGap.v
Unitary steps and biased-walk time bound	minimal/QuantumChairs.v, minimal/BrownianComputer.v
Universal floor, simulating systems	coq/kernel/nfi/UniversalCertificationCost.v, coq/kernel/nfi/HonestCostTracking.v
Quantitative form	coq/kernel/nfi/QuantitativeNoFI.v
Cost frameworks	coq/kernel/nfi/CostSemanticsComparison.v, coq/kernel/nfi/CostFrameworks.v
Exact pricing	coq/kernel/nfi/CommitmentPredicateAdequacy.v
Permanent certificates, merge pricing	coq/kernel/nfi/PermanentCertification.v, coq/kernel/nfi/PermanentRecordPricing.v
Which finite premises are needed	coq/kernel/foundation/NecFFloor.v, coq/kernel/foundation/NecFSqueeze.v
Entropy and heat	coq/kernel/nfi/PermanentCertificationEntropy.v
Eight-state instance	coq/kernel/nfi/FiniteCertMachine.v
No intrinsic joule value	coq/kernel/nfi/PricingPhysicsAudit.v
Shadow pricing	coq/kernel/nfi/ShadowPricing.v
Narrowing	coq/kernel/nfi/KnowledgeNarrowing.v
Window adequacy, joint floors	coq/kernel/frontier/ObservationPolicy.v
Structural undecidability	coq/kernel/nfi/StructuralUndecidability.v, coq/kernel/foundation/NatSubstrateInstance.v, coq/kernel/foundation/Substrate.v
L recursion and Rice	coq/kernel/foundation/LRecursion.v

Record axis	coq/kernel/foundation/StructuralCore.v, coq/kernel/foundation/StructuralCoreAnyBase.v, coq/kernel/foundation/StructuralRecordAxis.v, coq/kernel/foundation/RecordAxisDiscrimination.v, coq/kernel/foundation/CrossBaseGranularityCore.v, coq/kernel/foundation/CrossBaseGranularity.v, coq/kernel/foundation/CrossBaseGranularityRAM.v, coq/kernel/foundation/CrossBaseGranularityL.v
Latches, merges and windows on the axis	coq/kernel/foundation/AxLatch.v, coq/kernel/foundation/AxLatch2.v, coq/kernel/foundation/AxMerge.v, coq/kernel/foundation/AxWindow.v, coq/kernel/foundation/AxInfinite.v
Growing and probabilistic records	coq/kernel/foundation/GrowingRecord.v, coq/kernel/foundation/ProbabilisticRecord.v
Calorimeter	coq/kernel/thermodynamic/CalorimeterProtocol.v
External models	coq/kernel/reductions/RFC9162Merkle.v, coq/kernel/reductions/NeculaPCC.v, coq/kernel/reductions/TPMQuoteAuthenticity.v, coq/kernel/reductions/CasperFFG.v
Pointer criterion	coq/kernel/frontier/PointerObservable.v, coq/kernel/frontier/RecordProliferationSurvey.v, coq/kernel/frontier/EcosystemGame.v, coq/kernel/frontier/ PointerObservableCounterexamples.v

Appendix B

Epistemological Classification

Result	Status	Key premise
The definition and the price in the move		
The member predicate; a flat reading obeys the toll and is not a member	**(S)**	Any preorder; flat means no step leaves the down-set
The latch is the smallest Thiele machine	**(S)**	Two states, one move priced 1; any member has two readings and a priced move
The history lift: a machine with a run that moves is the shadow of a member	**(S)**	A Boolean equality test on states; a run ending off its start; a finite base for the least price
The floor is the toll	**(S)**	A preorder of records; nothing else
Thresholds are one-bit machines	**(S)**	A2; antisymmetry for the thresholds to determine the record
Universal floor	**(S)**	A2 on the two-point order
All readings and free moves	**(S)**	Decidable state equality; one schedule for every Boolean reading
Least instruction price	**(S)**	Fixed dynamics and reading; finite covering list
Least unit lift price	**(S)**	An event witness for every mark; all-state toll
A2 from merge pricing	**(S)**	Finite states, permanent certificate, merge price
Shadow pricing	**(S)**	A colliding pair in the window
What stays		
Free reading preservation	**(S)**	A2 and global permanence
Record axis is a latch	**(S)**	Base-driven record
What a window forgets		
Canonical future equivalence	**(S)**	Fixed deterministic operations and observations
The room a kept distinction needs		
Merge or fresh states	**(S)**	Total deterministic step, permanent raise
Sharp retention bound	**(S)**	Finite implementation, distinct representatives, reset and injectivity through the specified horizon
Stochastic and unitary holding	**(S)**	Finite stochastic kernel with column constraints, or finite unitary dynamics and invariant projection
What no inside procedure decides		
Structural undecidability	**(S)**	Recursion theorem of the substrate
L recursion and Rice	**(S)**	L's reduction rules
The world		
Biased-walk heat against time	**(S)**	Specified biased-walk entropy formula; positive speed below one
Heat floor	**(C)**	`landauer_heat`, finite states, permanent certificate, the uniform distribution on the states in play

No intrinsic joule value	**(R)**	None beyond the ledger
Calorimeter heat scale	**(R)**	Fixed dynamics
Pointer criterion	Thesis	Observer model